Java Concurrency & Multithreading Interview Notes

1. Process vs Thread

Process

- A process is a program in execution.
- Each process has its own memory space.
- Processes are isolated from each other.
- Communication between processes is slower (IPC, sockets, pipes).

Thread

- A thread is the smallest unit of execution inside a process.
- Multiple threads belong to the same process.
- Threads share resources, making communication faster.

Shared among Threads

- Heap Memory
- Static Variables
- Method Area (Metaspace)
- Open Files
- Socket Connections

Private to Each Thread

- Stack Memory
- Program Counter (PC Register)
- Local Variables

Interview Answer

> A process is an independent program running in the operating system, whereas a thread is the smallest unit of execution within a process. Threads share heap memory and resources but have their own stack and program counter. Threads are lightweight and faster than processes because thread creation and context switching are cheaper.

---

2. Race Condition

A race condition occurs when multiple threads access and modify shared data simultaneously without proper synchronization.

Example

```
class Counter {
    int count = 0;

    public void increment() {
        count++;
    }
}
```

Two threads calling `increment()` may produce incorrect results because `count++` is not atomic.

---

3. Why count++ is Not Atomic

Internally,

Read count
↓

Increment

↓

Write count

Example

count = 10

Thread1 reads 10
Thread2 reads 10

Thread1 writes 11
Thread2 writes 11

Final value = 11

Expected = 12

This is called a **Lost Update**.

---

4. How to Fix Race Condition

synchronized

```
synchronized void increment() {
    count++;
}
```

ReentrantLock

```
lock.lock();

try {
    count++;
} finally {
    lock.unlock();
}
```

AtomicInteger

```
AtomicInteger count = new AtomicInteger();

count.incrementAndGet();
```

Best for counters.

---

5. Compare-And-Swap (CAS)

CAS = Compare And Swap

Used internally by:

- AtomicInteger
- AtomicLong
- AtomicReference

Working:

Current Value = 10

Expected = 10

New Value = 11

If current == expected

Update succeeds

Else retry

Advantages

- No locking
- High performance
- Lock-free programming

---

6. volatile vs synchronized

volatile

- Guarantees visibility.
- Reads latest value from main memory.
- Does NOT provide atomicity.
- No locking.

Example

volatile boolean running = true;

Used for flags.

---

synchronized

- Guarantees visibility.
- Guarantees mutual exclusion.
- Prevents race conditions.
- Uses intrinsic monitor lock.

Example

```
synchronized void update() {
   balance++;
}
```

Difference

volatile	synchronized
Visibility only	Visibility + Atomicity
No Lock	Uses Lock
No Mutual Exclusion	Mutual Exclusion

---

7. wait() vs sleep()

sleep()

- Thread class
- Pauses execution
- Does NOT release lock

```
Thread.sleep(2000);
```

---

wait()

- Object class
- Releases monitor lock
- Used for thread communication
- Must be inside synchronized block

```
synchronized(lock) {
   lock.wait();
}
```

---

notify()

Wakes one waiting thread.

lock.notify();

---

notifyAll()

Wakes all waiting threads.

lock.notifyAll();

---

8. Why wait(), notify() belong to Object?

Because synchronization happens on an object's monitor (intrinsic lock).

Every Java object has a monitor.

A thread acquires the object's monitor before entering a synchronized block.

If these methods were in Thread, Java wouldn't know which object's lock to release.

---

9. Deadlock

Deadlock occurs when two or more threads wait forever for each other's locks.

Bank Transfer Example

Thread1

Lock Account A

Waiting for Account B

Thread2

Lock Account B

Waiting for Account A

Both wait forever.

Prevention

- Acquire locks in same order.
- Use tryLock().
- Avoid nested locks.
- Use lock timeout.

---

10. Executor Framework

Problems with new Thread()

- Expensive thread creation
- Memory overhead
- Too many threads
- Poor resource management

Executor Framework solves this using Thread Pool.

Thread Pool

Task

↓

Queue

↓

Worker Thread

↓

Execute

↓

Return to Pool

Threads are reused instead of recreated.

---

11. Executor Hierarchy

Executor

Basic interface.

execute(Runnable task);

ExecutorService

Adds

- submit()
- shutdown()
- invokeAll()
- Future support

ScheduledExecutorService

Used for delayed and periodic tasks.

scheduler.schedule(task,5,TimeUnit.SECONDS);

scheduler.scheduleAtFixedRate(task,0,10,TimeUnit.SECONDS);

Example

- Cache cleanup
- Heartbeat
- Daily reports

---

12. ThreadPoolExecutor

Example

Core = 2

Max = 4

Queue = 2

Tasks = 10

Execution

Task1 -> Thread1

Task2 -> Thread2

Task3 -> Queue

Task4 -> Queue

Task5 -> Thread3

Task6 -> Thread4

Task7 -> Rejected

Task8 -> Rejected

Task9 -> Rejected

Task10 -> Rejected

---

13. RejectedExecutionHandler

AbortPolicy

Default

Throws

RejectedExecutionException

CallerRunsPolicy

Caller thread executes task.

Useful for backpressure.

DiscardPolicy

Silently discards new task.

DiscardOldestPolicy

Removes oldest queued task.

Adds new task.

---

14. HashMap vs ConcurrentHashMap

HashMap

- Not thread-safe
- Allows one null key
- Allows multiple null values

ConcurrentHashMap

- Thread-safe
- Java 8+: CAS + bucket-level locking
- Multiple reads simultaneously
- Only affected bucket is locked during write
- No null key
- No null value

Hashtable

- Thread-safe
- Synchronizes every method
- Entire table locked
- Poor concurrency
- No null key
- No null value

---

15. CompletableFuture

Used for asynchronous programming.

Advantages over Future

- Non-blocking
- Chaining
- Combining multiple tasks
- Better exception handling

Future Limitation

future.get();

Blocks caller thread.

---

thenApply()

Transforms result.

```
CompletableFuture.supplyAsync(() -> "Neelu")
        .thenApply(String::toUpperCase);
```

---

thenCompose()

Chains dependent async tasks.

Create Order

↓

Get OrderId

↓

Call Payment API

```
future.thenCompose(orderId ->
        paymentService.pay(orderId));
```

---

thenCombine()

Combines two independent async tasks.

User Service

+

Inventory Service

↓

Combine

```
userFuture.thenCombine(inventoryFuture,
        (u, i) -> new Order(u, i));
```

---

allOf()

Runs multiple tasks in parallel.

```
CompletableFuture.allOf(
        userFuture,
        inventoryFuture,
        paymentFuture
).join();
```

Example

User Service

Inventory Service

Payment Service

↓

Run Parallel

↓

Wait for All

↓

Continue

---

16. Real Project Example

Suppose Order Service receives a request.

Run these validations in parallel:

- Inventory Service
- Pincode Validation
- Customer Eligibility

Using

CompletableFuture.allOf(...)

If all validations pass

↓

Create Order

Else

↓

Return Validation Error

This reduces response time because independent tasks execute concurrently.

Interview Tips

- Speak slowly and confidently.
- First explain the concept.
- Then explain why it is needed.
- Give a real-world example.
- Mention where you used it in your project.
- If you don't know something, say:
  "I haven't used it extensively in production, but I understand the concept and where it is applicable."

This sounds much better than guessing.

Question 1

Suppose I have this code:

```
ExecutorService executor = Executors.newFixedThreadPool(5);


for(int i=0;i<100;i++){
    executor.submit(() -> process());
}
```

Questions:

1. **What happens internally when `submit()` is called?**
2. **Where are the remaining 95 tasks stored?**
3. **How does a thread pick the next task?**
4. **Why don't 100 threads get created?**

Answer : When `submit()` is called, the task is wrapped in a `FutureTask` because `submit()` returns a `Future`. The `ThreadPoolExecutor` checks whether a worker thread is available. If all five workers are busy, the task is placed into the `LinkedBlockingQueue`. Each worker repeatedly takes tasks from the queue and executes them. Using a fixed thread pool avoids the overhead of creating and destroying threads for every request, reduces memory usage, and minimizes CPU time spent on context switching.

Question:

What is the difference between `execute()` and `submit()`?

Answer : `execute()` is used for fire-and-forget tasks. It accepts only `Runnable`, returns `void`, and doesn't provide a result. `submit()` accepts both `Runnable` and `Callable`, returns a `Future`, allows you to retrieve the result or monitor task completion, and propagates exceptions through `Future.get()`."

Next Question (Very Common)

Imagine this code:

ExecutorService executor =

Executors.newFixedThreadPool(5);

Question:

👉 **Why do experienced Java developers recommend creating a `ThreadPoolExecutor` directly instead of using `Executors.newFixedThreadPool()`?**

> "`Executors.newFixedThreadPool()` is a convenience factory method that internally creates a `ThreadPoolExecutor` with default settings. One major drawback is that it uses an **unbounded `LinkedBlockingQueue`**. If tasks are produced faster than they are consumed, the queue can grow indefinitely, potentially leading to high memory usage or even an `OutOfMemoryError`."
>
> "By creating a `ThreadPoolExecutor` directly, I have full control over:
>
> - Core pool size
> - Maximum pool size
> - Queue type and capacity
> - Keep-alive time
> - Thread factory
> - Rejected execution handler"

Interview Question 2 (Java Concurrency)

Suppose you have this code:

Map<Integer, String> map = new HashMap<>();

```
Thread t1 = new Thread(() -> {
    map.put(1, "A");
});

Thread t2 = new Thread(() -> {
    map.put(2, "B");
});

t1.start();
t2.start();
```

Questions:

1. **Why is `HashMap` not thread-safe?**
2. **What problems can occur if multiple threads read and write to a `HashMap` simultaneously?**
3. **Why is `ConcurrentHashMap` thread-safe?**
4. **Can two threads write to a `ConcurrentHashMap` at the same time? Explain how.**

HashMap is not thread-safe because it performs no synchronization. If multiple threads modify it concurrently, we can get race conditions, lost updates, inconsistent reads, and even structural corruption. `Collections.synchronizedMap()` makes it thread-safe by synchronizing every operation, but it locks the entire map, which limits scalability. In Java 8+, `ConcurrentHashMap` achieves much better concurrency by using CAS for many updates and locking only the affected bucket when necessary, allowing multiple threads to operate on different buckets simultaneously. Reads are generally lock-free, which provides much better throughput.

Question:

Why should we use `compute()` or `computeIfAbsent()` instead of writing code like this?

Follow-up Question (Very Common)

Now the interviewer may ask:

> **"If `AtomicInteger` is so good, why do we still need `synchronized` or `ReentrantLock`?"**

This is a classic senior-level question.

Think about it.

`AtomicInteger` is great for:

count++;

But suppose you have:

withdraw();

updateDatabase();

sendNotification();

Can one `AtomicInteger` make **all three operations** atomic together?

👉 **Answer this question.**

This is where interviewers distinguish between someone who knows APIs and someone who understands concurrency design.

atomic integer provide synchroisation on numbers there are multiple lines of code we should use shronised block or retenter lock

Excellent! 👏 That's the right idea. I'd give this **9.5/10**.

The only thing I'd improve is the terminology.

Interview Tip

If the interviewer asks:

When do you use `AtomicInteger` and when do you use `synchronized`?

A strong answer is:

> `AtomicInteger` is suitable for a **single shared variable** and atomic operations such as increment, decrement, or compare-and-set. If I need to make multiple operations execute as one atomic unit, I would use `synchronized` or `ReentrantLock`.
>
> "I use `AtomicInteger` when I need atomic operations on a single shared variable because it's lock-free and performs well under contention. If I need to protect a business transaction involving multiple variables or multiple statements that must execute together, I use `synchronized` or `ReentrantLock`."

What is the difference between `join()` and `get()`?

> `get()` comes from the `Future` interface and throws checked exceptions like `InterruptedException` and `ExecutionException`. `join()` is provided by `CompletableFuture`, returns the result in the same way, but throws unchecked `CompletionException`, making asynchronous code cleaner and easier to write.